Fuzzing Guiado por Contrato para APIs REST

Title: Contract-Guided Fuzzing for REST APIs

Levi da Rosa Gomes (aluno)[1], **Fábio Austrian Zanin (orientador)**[1]

[1] Universidade Regional Integrada do Alto Uruguai e das Missões
Departamento de Engenharias e Ciência da Computação
Caixa Postal 743 – 99.709-910 – Erechim – RS – Brasil

103495@uricer.edu.br, faz@uricer.edu.br

Abstract. *Divergences between REST APIs and their OpenAPI contracts undermine the integration between systems. This work presents a black-box, contract-guided fuzzer that generates valid and mutated requests, checks the responses against the contract, groups equivalent findings, and assigns each one a triage priority through fuzzy inference. The evaluation used five open-source APIs in two stages. In the first, four APIs guided the revision of the prioritization model; once frozen, the model was applied to a fifth API, Meilisearch, and compared with a reference produced blindly by three instances of a language model (GPT-5.6 Luna), agreeing on 120 of the 122 behavior groups assessed (98%), against 4 (3%) for a rule based only on the HTTP status. In the second stage, 25 campaigns, five per API, executed all 7,175 planned requests; grouping reduced 4,000 findings to 2,460 groups (38.5%), and 441 of the 570 distinct behaviors (77%) reappeared in all five repetitions of their API. The results indicate that the approach organizes traceable evidence for triage, without confirming vulnerabilities or estimating real impact.*
Keywords *REST APIs, OpenAPI, Fuzzing, Contract Testing, Fuzzy Logic.*

Resumo. *Divergências entre APIs REST e seus contratos OpenAPI comprometem a integração entre sistemas. Este trabalho apresenta um fuzzer de caixa-preta guiado por contrato que gera requisições válidas e mutadas, compara as respostas com o contrato, agrupa achados equivalentes e atribui a cada um uma prioridade de triagem por inferência fuzzy. A avaliação usou cinco APIs de código aberto em duas etapas. Na primeira, quatro APIs orientaram a revisão do modelo de priorização; depois de congelado, o modelo foi aplicado a uma quinta API, o Meilisearch, e comparado com uma referência produzida às cegas por três instâncias de um modelo de linguagem (GPT-5.6 Luna), concordando em 120 dos 122 grupos de comportamento avaliados (98%), contra 4 (3%) de uma regra baseada apenas no status HTTP. Na segunda etapa, 25 campanhas, cinco por API, executaram as 7.175 requisições planejadas; o agrupamento reduziu 4.000 achados a 2.460 grupos (38,5%), e 441 dos 570 comportamentos distintos (77%) reapareceram nas cinco repetições da respectiva API. Os resultados indicam que a abordagem organiza evidências rastreáveis para triagem, sem confirmar vulnerabilidades nem estimar impacto real.*
Palavras-Chave *APIs REST, OpenAPI, Fuzzing, Teste de Contrato, Lógica Fuzzy.*

1. Introdução

APIs REST tornaram-se parte recorrente da infraestrutura de sistemas modernos, conectando serviços, aplicações *web* e dispositivos móveis [Golmohammadi et al. 2023]. Nesse cenário,

falhas de robustez e inconsistências entre o comportamento da API e o contrato publicado afetam a confiabilidade das integrações e o custo de manutenção. Testes funcionais tradicionais validam cenários esperados, mas podem deixar de exercitar valores limite, formatos inválidos e combinações pouco usuais de parâmetros.

Contratos como a especificação OpenAPI descrevem operações, estruturas, tipos, restrições e respostas esperadas. Eles podem orientar a geração sistemática de entradas válidas e inválidas e servir como oráculo parcial para identificar divergências entre especificação e implementação. Essa abordagem permite observar a API externamente, sem instrumentação de código, em um contexto *black-box* relacionado à noção de contrato de software [Meyer 1992].

Este trabalho apresenta uma ferramenta de *fuzzing* automatizado para APIs REST, implementada em Python e orientada por um subconjunto de OpenAPI. O fluxo interpreta o contrato, gera casos-base e mutações, materializa um plano de execução, envia requisições, confronta as respostas com o contrato e produz artefatos rastreáveis. Um sistema de inferência *fuzzy* (FIS, do inglês *fuzzy inference system*), do tipo Mamdani, atribui prioridade técnica às anomalias reportadas depois da classificação contratual.

A questão central é: *Quais contribuições e limitações a combinação de fuzzing black-box orientado por OpenAPI, deduplicação e priorização fuzzy apresenta para a triagem de divergências em APIs REST?*

O objetivo geral é implementar essa combinação e avaliá-la em APIs de código aberto executadas localmente. Os objetivos específicos correspondem às três dimensões da avaliação: (i) verificar a execução e a completude das evidências de anomalias contratuais e comportamentais; (ii) medir a estabilidade dos comportamentos entre campanhas; e (iii) medir a redução por deduplicação e avaliar a coerência da prioridade técnica para triagem.

Quatro APIs foram utilizadas como pilotos de desenvolvimento, permitindo auditar a configuração inicial e formular um modelo revisado. Após o congelamento das regras, uma quinta API foi empregada como *holdout* externo. Nessa validação externa, a prioridade calculada foi comparada com uma referência construída às cegas: três instâncias do modelo de linguagem GPT-5.6 Luna, executadas separadamente e sem acesso ao escore do FIS, classificaram cada comportamento segundo os mesmos critérios, e uma quarta instância decidiu os casos sem maioria. Em seguida, uma coleta final reuniu cinco campanhas de cada uma das cinco aplicações para avaliar completude, deduplicação e estabilidade dos achados.

O escopo limita-se a APIs REST descritas em OpenAPI. Não são abrangidos sequenciamento geral com estado, fluxos completos de OAuth2, avaliação de carga, cobertura de código ou confirmação automática de vulnerabilidades. Um achado pode representar *contract drift*, incompatibilidade funcional, robustez ou comportamento de controle de acesso e ainda exigir interpretação antes de ser tratado como defeito. Integração contínua é uma motivação, mas não é implementada nem avaliada.

O restante deste artigo está organizado da seguinte forma. A Seção 2 apresenta os conceitos necessários. A Seção 3 posiciona a proposta diante das ferramentas relacionadas. A Seção 4 descreve o desenho da avaliação, e a Seção 5 apresenta a ferramenta. A Seção 6 apresenta os resultados da avaliação da priorização e os da coleta final. Por fim, a Seção 7 sintetiza as contribuições e limitações observadas.

2. Fundamentação Teórica

Esta seção reúne os conceitos necessários para compreender a proposta sem antecipar o protocolo experimental. O encadeamento adotado distingue três etapas: o contrato fornece uma descrição processável e expectativas parciais; o teste *black-box* produz observações e evidências; e a inferência *fuzzy* ordena os achados já detectados. Essa separação é importante porque gerar uma requisição, reconhecer uma divergência e atribuir prioridade são problemas relacionados, mas não equivalentes.

2.1. REST, semântica HTTP e contratos processáveis

REST é um estilo arquitetural para sistemas distribuídos, definido por restrições como identificação de recursos, interface uniforme, comunicação sem estado entre requisições e transferência de representações [Fielding 2000]. HTTP fornece a semântica observável da interação: métodos expressam a intenção da requisição, enquanto códigos de status e cabeçalhos comunicam características da resposta [Fielding et al. 2022]. Uma API usar HTTP e JSON, portanto, não basta para demonstrar que ela satisfaz todas as restrições de REST; esses elementos são apenas parte da interface que um teste externo consegue observar.

A OpenAPI Specification (OAS) descreve APIs HTTP de forma independente de linguagem. Um documento pode declarar caminhos, operações, parâmetros, corpos, respostas e esquemas de segurança [OpenAPI Initiative 2021]. Em OpenAPI 3.1, a descrição dos dados se alinha ao JSON Schema, cujo vocabulário permite formular asserções estruturais com palavras-chave como `type`, `required`, `enum`, limites numéricos, padrões e propriedades [JSON Schema 2022]. Essas declarações tornam uma parte do acordo entre cliente e serviço legível por ferramentas.

Há uma analogia útil com *Design by Contract*: precondições expressam obrigações do cliente, pós-condições expressam garantias do fornecedor e invariantes preservam propriedades do objeto [Meyer 1992]. A analogia não estabelece equivalência formal. OpenAPI não expressa necessariamente invariantes, dependências entre chamadas ou todas as regras de negócio; além disso, o documento publicado pode estar incompleto ou desatualizado.

Neste trabalho, o contrato é uma descrição processável do comportamento esperado e não uma especificação completa da aplicação. O *fuzzer* interpreta um subconjunto de OpenAPI para construir requisições e conferir propriedades observáveis. Logo, toda conclusão deve ser delimitada tanto pelo que o contrato declara quanto pelo que a ferramenta efetivamente suporta.

2.2. *Fuzzing black-box*, oráculos e evidência

Fuzzing automatiza a produção e a execução de muitas entradas [Golmohammadi et al. 2023]. Uma estratégia geracional constrói casos a partir de um modelo, enquanto uma estratégia mutacional transforma casos existentes. Em APIs REST, o contrato permite preservar a estrutura de parâmetros e corpos ao gerar casos potencialmente válidos e também orientar teste negativo, no qual obrigatoriedade, tipo, faixa ou outra restrição é deliberadamente violada. RESTest, por exemplo, distingue casos nominais de casos defeituosos e explicita oráculos diferentes para requisições que deveriam ser aceitas ou rejeitadas [Martin-Lopez et al. 2020]; RestTestGen concretiza a mesma separação em estratégias nominal e de erro, com mutadores próprios [Corradini et al. 2022].

No modo *black-box*, a geração e a decisão não recebem cobertura de código nem instrumentação interna. Contrato, configuração, requisições e respostas são as fontes de evidência disponíveis. Isso amplia a aplicabilidade a serviços cujo código não está acessível, mas limita a capacidade de explicar causas internas. Autenticação, estado persistente, dados realistas e dependências entre operações continuam sendo obstáculos relevantes [Golmohammadi et al. 2023]. RESTler mostra uma resposta específica a parte desse problema: infere relações produtor–consumidor do contrato, preserva valores obtidos nas respostas e explora sequências de requisições com realimentação da execução [Atlidakis et al. 2019]. Essa exploração com estado é diferente de apenas repetir operações isoladas.

Um oráculo de teste decide se a observação satisfaz alguma expectativa. Status de servidor, status não declarado, conformidade do corpo com o esquema e aceitação de uma entrada inválida são exemplos de propriedades verificáveis a partir da interface. Nenhuma delas, isoladamente, comprova que toda a operação está correta. O contrato atua, portanto, como *oráculo parcial*: permite detectar contradições relativas às declarações disponíveis, mas não cobre efeitos de negócio, intenção do fornecedor, impacto ou segurança. Estudos da área mostram que ferramentas baseadas em OpenAPI adotam combinações distintas de geração e oráculos, e que a incompletude dos esquemas limita a exploração *black-box* [Golmohammadi et al. 2023, Zhang e Arcuri 2023].

Uma observação só é útil para análise posterior se permanecer ligada às condições que a produziram. RESTler associa o erro à sequência de requisições e agrupa ocorrências provavelmente redundantes; Schemathesis combina redução do caso que falha com deduplicação de erros; e RestTestGen pode registrar em JSON as requisições, respostas e avaliações dos oráculos [Atlidakis et al. 2019, Hatfield-Dodds e Dygalo 2021, Corradini et al. 2022]. Esses mecanismos não são idênticos, mas evidenciam três necessidades distintas: *rastreabilidade* relaciona a observação à sua origem; *reprodução* preserva informação suficiente para provocar novamente o comportamento; e *deduplicação* reduz ocorrências equivalentes para a triagem. Um grupo deduplicado não corresponde automaticamente a uma causa raiz.

O projeto adota esse vocabulário de forma operacional. Uma ocorrência é o resultado individual de um caso executado. Uma divergência é uma propriedade observada que contradiz uma expectativa interpretada do contrato ou de um detector. Um achado é a evidência estruturada dessa divergência, vinculada ao caso e à resposta. Assinaturas e famílias agrupam padrões observáveis para evitar que repetições dominem a análise. Em um possível ciclo de integração contínua, um resultado útil para triagem deve ser produzido sob orçamento conhecido, manter essa cadeia de evidências, separar campanha incompleta de comportamento do alvo e apresentar volume tratável. Esses são critérios de uso adotados pelo estudo.

2.3. Inferência *fuzzy* e prioridade técnica

Em um conjunto clássico, a pertinência é binária. Em um conjunto *fuzzy*, uma função associa cada elemento a um grau no intervalo entre zero e um [Zadeh 1965]. Esse grau representa o quanto a observação pertence a um conceito gradual definido pelo modelo; não deve ser interpretado como probabilidade. Termos linguísticos como baixa, média e alta podem, assim, representar regiões sobrepostas de uma variável.

O sistema implementado organiza a inferência em quatro passos conceituais: calcula os graus de pertinência, avalia regras linguísticas, agrega as conclusões ativadas e desfuzzifica o conjunto resultante em um valor numérico. Ele adota a abordagem linguística de Mamdani, cujas

regras têm a forma “se ... então ...” e mantêm consequentes *fuzzy* [Mamdani e Assilian 1975]. As funções de pertinência, regras, operadores e o método de desfuzzificação são escolhas do projeto; o trabalho de Mamdani não determina os parâmetros adequados para este domínio.

Prioridade também exige delimitação. Na literatura de regressão, priorizar casos de teste significa alterar sua ordem para atingir antes um objetivo, como revelar falhas [Elbaum et al. 2002]; abordagens *fuzzy* já foram empregadas para ordenar esses casos antes da execução [Alakeel 2014]. O problema deste projeto ocorre depois da execução: os analisadores já produziram achados, e o sistema Mamdani ordena quais evidências merecem inspeção primeiro.

Por isso, *prioridade técnica* designa neste TCC a urgência relativa de triagem segundo os sinais presentes nos artefatos. Ela não equivale a severidade universal, impacto de negócio, explorabilidade ou probabilidade de vulnerabilidade. Uma vulnerabilidade requer que condições e consequências de segurança sejam confirmadas; uma divergência contratual pode ser relevante para compatibilidade ou robustez sem constituir vulnerabilidade. O FIS não detecta a divergência nem substitui essa análise: ele recebe achados já classificados e produz uma ordenação interpretável. Suas entradas, funções e regras são escolhas do projeto e precisam ser descritas e avaliadas empiricamente, em vez de serem tratadas como consequência automática da literatura *fuzzy*.

3. Trabalhos Relacionados

Esta seção compara estratégias de teste automatizado de APIs REST que usam um contrato como entrada. A análise é exclusivamente bibliográfica: nenhuma dessas ferramentas foi executada como linha de base experimental. O objetivo não é escolher uma vencedora, mas identificar como diferentes trabalhos tratam geração, estado, oráculos e apresentação dos resultados, preservando os limites das versões e dos experimentos descritos nas fontes.

Foram selecionados Schemathesis, RESTler, RESTest e RestTestGen porque possuem fontes primárias acessíveis e cobrem decisões diretamente relacionadas ao projeto. A comparação observa: formato de contrato e acesso *black-box* ou *white-box*; geração válida, inválida e com estado; oráculos e classes de falha; artefatos, reprodução e deduplicação; priorização de resultados; e relação declarada com integração contínua.

3.1. Estratégias de geração e tratamento de estado

As quatro abordagens partem de uma descrição de API, mas não constroem o espaço de testes da mesma forma. RESTler compila Swagger/OpenAPI em uma gramática, infere dependências produtor–consumidor e usa valores extraídos das respostas para formar sequências com estado. A execução fornece realimentação para descartar sequências inviáveis e aprofundar estados alcançáveis [Atlidakis et al. 2019]. Seu ponto central é a ordem entre operações e a propagação de recursos dinâmicos.

RESTest trabalha no nível de uma operação por caso. Ele gera entradas nominais e variantes inválidas e pode enriquecer OpenAPI com IDL4OAS para expressar dependências entre parâmetros que o contrato comum não contém. A geração baseada em restrições tenta satisfazer ou violar deliberadamente essas dependências [Martin-Lopez et al. 2020]. Assim, sua noção de dependência está principalmente dentro da requisição, enquanto a exploração com estado entre operações não é o foco do artigo-base.

Schemathesis deriva geradores e propriedades de OpenAPI ou GraphQL por meio de teste baseado em propriedades. A fonte descreve geração de entradas válidas e negativas, redução automática de casos que falham e testes em máquina de estados, estes dependentes de relações entre operações como os `links` de OpenAPI [Hatfield-Dodds e Dygalo 2021]. RestTestGen, por sua vez, oferece componentes extensíveis para análise, modelos de dados, provedores de valores, mutadores, ordenação, execução e oráculos. Sua estratégia nominal ordena operações a partir de um grafo de dependências e reutiliza valores observados; a estratégia de erro aplica mutações a interações nominais bem-sucedidas [Corradini et al. 2022].

Essas diferenças produzem custos e alcances distintos. No estudo comparativo de Corradini et al., RestTestGen, RESTler, bBOXRT e RESTest foram avaliados em 14 serviços com versões e configurações de 2021; Schemathesis não integrou esse corpus. Robustez da ferramenta, enumeração de sequências e diversidade de dados competiram pelo mesmo orçamento de teste [Corradini et al. 2021]. Os resultados ajudam a explicar os compromissos, mas não estabelecem superioridade atual ou universal de uma estratégia.

3.2. Oráculos, evidências e triagem

As fontes também divergem no que consideram um resultado relevante. O artigo inicial do RESTler trata respostas 5xx como falhas observáveis e associa cada uma à sequência que a produziu. Como ocorrências repetidas são frequentes, propõe *bucketization* por sufixos de sequências para facilitar a inspeção; os autores relatam que as falhas do estudo puderam ser reproduzidas [Atlidakis et al. 2019]. Esse agrupamento é específico ao modelo do RESTler e não prova causa raiz comum.

RESTest explicita cinco oráculos: ausência de 5xx, conformidade da resposta com OAS, rejeição de violações de parâmetros, rejeição de violações de dependências e não rejeição de requisições consideradas válidas. Também gera testes Java executáveis e oferece execução *offline* ou *online*; a fonte apresenta esta última como aplicável a um ambiente de integração contínua [Martin-Lopez et al. 2020]. O artigo não descreve uma ordenação pós-execução ou uma deduplicação geral dos achados.

Schemathesis deriva propriedades de semânticas HTTP e do esquema, incluindo status não declarado, corpo incompatível e rejeição de entrada inválida. O mecanismo subjacente de teste baseado em propriedades fornece redução e deduplicação de erros, e os testes derivados podem integrar uma suíte existente ou ser executados diretamente [Hatfield-Dodds e Dygalo 2021]. RestTestGen separa oráculos de status e esquema, registra sequências, requisições, respostas e resultados dos oráculos em relatórios JSON e pode materializar casos com REST-assured [Corradini et al. 2022]. Na fonte-base, nenhuma das duas abordagens apresenta um classificador geral de prioridade técnica dos achados.

Para tornar os limites verificáveis, a Tabela 1 resume apenas capacidades documentadas nas fontes citadas. “Não demonstrado” significa que o aspecto não foi caracterizado no trabalho consultado, e não que seja impossível ou esteja ausente de toda versão posterior.

3.3. Posicionamento da proposta

A comparação evidencia que geração por OpenAPI, casos negativos, validação de status ou esquema, sequências com estado e redução de resultados já aparecem em trabalhos anteriores.

Tabela 1. Comparação bibliográfica das estratégias selecionadas, conforme as fontes citadas no texto

Ferramenta	Contrato, acesso e geração	Estado e casos negativos	Oráculos e evidências	Priorização e CI
Schemathesis	OpenAPI/GraphQL; *black-box* ou *white-box* em processo; geração baseada em propriedades	Entradas válidas e negativas; máquina de estados dependente das relações disponíveis	Propriedades de HTTP/esquema; redução automática e deduplicação de erros no mecanismo subjacente	Sem ordenação pós-execução demonstrada; integrável a suítes de teste
RESTler	Swagger/OpenAPI; *black-box*; gramática e dicionários	Sequências com estado, relações produtor–consumidor e valores mutados	5xx no oráculo-base; sequência preservada e *bucketization* de ocorrências	Busca prioriza exploração, não achados; CI não caracterizada na fonte
RESTest	OpenAPI com IDL4OAS opcional; *black-box*; geração aleatória ou por restrições	Uma operação por caso; entradas nominais e inválidas, inclusive dependências entre parâmetros	Cinco oráculos; testes executáveis; deduplicação não demonstrada	Sem ordenação de achados; execução *online* apresentada para CI
RestTestGen	OpenAPI; *black-box*; provedores de valores e mutadores extensíveis	Grafo de dependências, reuso de valores e estratégias nominal/de erro	Oráculos de status e esquema; relatórios JSON e testes REST-assured	Prioriza geração/ordem, não achados; CI não avaliada na fonte
Projeto deste TCC	Subconjunto de OpenAPI; *black-box*; caso-base e mutações tipadas	Níveis L0, L1 e L2; dependências com estado apenas parciais	Detectores contratuais; plano, respostas, manifestos e assinaturas deduplicadas	FIS Mamdani pós-execução; CI apenas como contexto de uso

Também mostra que "priorização" pode se referir a coisas diferentes: escolher operações e valores durante a geração, guiar a busca por sequências ou ordenar evidências depois que a execução terminou. O projeto atua no terceiro sentido.

A lacuna investigada não é a ausência de *fuzzers* de API nem a superioridade de uma técnica de geração. O estudo examina as contribuições e limitações de uma combinação específica: *fuzzing black-box* orientado por OpenAPI, evidências rastreáveis, deduplicação por comportamento observável e prioridade *fuzzy* pós-execução. Essa formulação é restrita ao corpus bibliográfico consultado e é avaliada nas aplicações do estudo. Como não há comparação experimental com as ferramentas desta seção, os resultados não autorizam afirmar maior cobertura, eficácia ou qualidade do projeto em relação a elas.

4. Metodologia

Para responder à questão de pesquisa, adota-se uma avaliação aplicada e experimental, baseada em execuções locais controladas. Esta seção define o papel das aplicações, a geração dos casos, as condições de repetição e as métricas usadas para analisar os artefatos produzidos. O estudo tem duas etapas. Na **Etapa 1 — avaliação da priorização**, quatro aplicações orientaram a revisão do FIS e uma quinta foi usada para validá-lo depois de congelado. Na **Etapa 2 — coleta final**, o modelo congelado foi executado em cinco campanhas de cada uma das cinco aplicações.

4.1. Desenho do Estudo e Papel das Aplicações

Na Etapa 1, VAmPI, Spring Petclinic REST, crAPI e RealWorld serviram para desenvolver a ferramenta e auditar a configuração inicial do FIS. O Meilisearch v1.45.1 só foi executado depois da revisão do modelo, sem participar do ajuste das regras.

As aplicações foram executadas localmente em versões fixadas e reiniciáveis. Contratos, imagens, configuração da ferramenta e procedimentos de inicialização foram registrados para separar mudanças do objeto avaliado de variações do ambiente.

A coleta final reuniu 25 campanhas novas, cinco por aplicação. Antes de cada campanha, a aplicação e seus dados foram reiniciados, para que o resultado não dependesse da execução anterior. Todas as campanhas usaram a mesma revisão do *fuzzer*, sem alterações locais, registrada nos manifestos. As campanhas do Meilisearch feitas na Etapa 1 não integram essas 25 execuções.

O objetivo das repetições não é gerar casos diferentes: com o mesmo contrato, revisão e configuração, o plano de requisições tende a ser o mesmo. As repetições verificam se as respostas e os achados se mantêm depois do reinício. Essa distinção é necessária porque a execução ainda pode variar: alguns executores geram credenciais automaticamente, e o estado da aplicação, o tempo de resposta ou uma falha transitória podem mudar entre campanhas.

4.2. Geração dos Testes e Níveis de Mutação

A geração começa com um caso compatível com as informações interpretadas do contrato. A partir dele, o motor aplica operadores tipados aos corpos JSON. Os parâmetros de caminho, consulta e cabeçalho recebem valores gerados, mas não participam desse motor de mutações. Três níveis organizam a exploração:

- **Nível 0 (L0) — caso-base:** executa a entrada gerada para satisfazer o contrato e estabelece a referência nominal da operação;
- **Nível 1 (L1) — mutação isolada:** aplica uma alteração por caso, permitindo relacionar a resposta com um operador específico;
- **Nível 2 (L2) — mutações combinadas:** combina duas alterações compatíveis para explorar interações, preservando orçamento próprio e os casos isolados de L1.

Essa progressão distingue comportamento nominal, efeito individual e interação entre mutações. O plano também inclui, quando habilitados, testes de operações protegidas sem credencial ou com credencial inválida. Limites de casos, concorrência, atraso e tempo limite são registrados como parte da configuração.

4.3. Protocolo Experimental e Estratégia de Avaliação

Cada campanha segue o fluxo contrato, plano, execução, análise e pós-processamento. Um manifesto registra identificador do experimento, API, repetição, revisão do código, estado Git, resumos criptográficos, ambiente, configuração, horários e contagens. O plano e os resultados são correlacionados por `case_id`.

As campanhas da coleta final usaram a configuração já exercitada nos pilotos. O executor percorreu as operações selecionadas depois das exclusões específicas de cada alvo e executou os níveis L0, L1 e L2. O limite configurado foi de 20 casos de mutação isolada por corpo no L1 e de 10 pares compatíveis no L2. Esses limites são fixos; a quantidade efetiva pode ser menor quando

o contrato oferece menos parâmetros, esquemas ou combinações aplicáveis. A concorrência foi de 1, o intervalo entre requisições de 250 ms e o tempo limite de 10 s. A configuração foi idêntica nas cinco campanhas de cada API. Entre alvos, variaram a autenticação e os caminhos excluídos, registrados nos manifestos.

Foram selecionadas 13 das 14 operações de VAmPI, 36 de 37 do Spring Petclinic REST, 44 de 44 da crAPI, 19 de 19 do RealWorld e 85 de 138 do Meilisearch. VAmPI excluiu a operação de recriação do banco; Petclinic excluiu o caminho `/oops`, definido para falhar. No Meilisearch, as exclusões abrangeram formatos de entrada não suportados, recursos ou integrações ausentes e operações que alterariam o índice usado como semente. Esses recortes limitam a cobertura do contrato. Como os contratos têm tamanhos diferentes, as APIs também produzem quantidades diferentes de casos e requisições; cada manifesto registra o número planejado e o executado.

Respostas 4xx e 5xx são observações do teste e não invalidam uma campanha. Uma campanha é válida quando o plano foi executado por completo, cada `case_id` tem resultado, os artefatos são legíveis e a versão do alvo, a revisão da ferramenta e a configuração estão registradas. Falha de inicialização, encerramento do alvo ou interrupção que deixe requisições sem resultado tornam a campanha inválida: ela é preservada com a justificativa, fica fora do agregado e, se a causa puder ser corrigida, é substituída por uma nova campanha, com outro identificador.

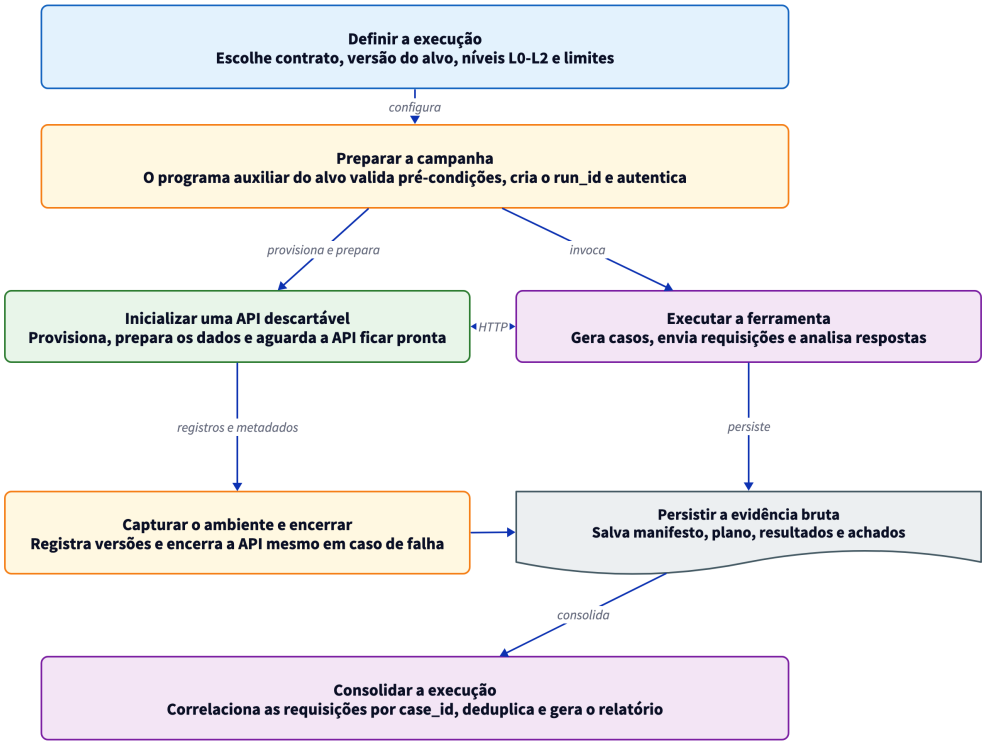


Figura 1. Ciclo de uma execução experimental isolada. O programa auxiliar específico prepara o alvo descartável e invoca a ferramenta. A ligação HTTP representa a troca de requisições e respostas; a evidência é completada com a proveniência do alvo antes da deduplicação e consolidação.

Para acompanhar uma execução sem misturar coisas diferentes, o estudo separa seis unidades:

- uma **requisição**, que é um caso enviado à API;
- uma **observação classificada**, emitida pelo analisador para uma requisição; rejeições esperadas entram nas contagens de execução, enquanto as demais observações candidatas são os achados para triagem;
- uma **assinatura estrita**, que agrupa, dentro de uma campanha, observações equivalentes por aplicação, operação, classificação, status, resposta normalizada e caminho do erro de validação;
- um **ID semântico**, que identifica esses grupos entre campanhas da mesma API. Preserva os componentes da assinatura estrita, com uma normalização adicional restrita aos campos `requestUid` e `processingTimeMs` nas respostas de `POST /multi-search` do Meilisearch. Essa normalização foi definida na Etapa 1, depois de se observar que esses dois campos mudavam a cada execução, e mantida sem alteração na Etapa 2;
- uma **campanha**, que é uma execução completa contra uma API;
- uma **família de comportamento**, usada na Etapa 1 para reunir assinaturas com a mesma operação, tipo de divergência, status e contexto do caso (caso-base, mutação inválida, mutação válida, credencial ausente ou credencial inválida).

Essa separação evita contar várias mutações que produziram a mesma resposta como falhas diferentes. Um grupo não corresponde necessariamente a uma causa raiz, e "semântico" designa apenas a regra de comparação descrita acima.

Para avaliar a prioridade na Etapa 1, foi construída uma referência independente do FIS. Cada família foi apresentada em uma ficha com o contrato, a requisição, a resposta e o número de repetições em que apareceu; a ficha omitia a classificação do analisador, o escore, a faixa e as regras do FIS. Três instâncias do modelo de linguagem GPT-5.6 Luna, executadas em contextos separados e sem acesso às avaliações umas das outras, aplicaram os mesmos critérios de avaliação. Os critérios pedem primeiro uma decisão de validade: a família é **sustentada** quando os fatos permitem afirmar uma anomalia técnica e atribuir prioridade, **não sustentada** quando o comportamento é compatível com o contrato e **indeterminada** quando falta evidência para decidir. Só as famílias sustentadas recebem prioridade: por exemplo, uma rejeição segura de entrada inválida tende a `LOW`, uma entrada inválida aceita a `MEDIUM` e a indisponibilidade repetida de uma operação válida a `HIGH`. Prevaleceu a decisão da maioria; quando não houve maioria, uma quarta instância do mesmo modelo decidiu, também sem acesso às avaliações anteriores ou ao FIS. As duas avaliações usaram os mesmos níveis e a mesma decisão de validade; os critérios foram revisados para a validação no Meilisearch, e o *prompt* usado nela está publicado no repositório do projeto[1], em `experiments/tcc-2026/prompt-referencia-cega.md`.

Como as quatro instâncias são do mesmo modelo, seus julgamentos são correlacionados, e a referência não substitui a avaliação de especialistas humanos. Como esses critérios também orientaram a revisão do FIS, a comparação mede se o FIS os reproduz em dados que não participaram do ajuste, e não se a prioridade está correta em sentido absoluto.

Na Etapa 1, o FIS é comparado com uma linha de base que usa apenas o desvio do status HTTP, o sinal que dominava o modelo inicial: status documentado recebe `LOW`, status

[1] `https://github.com/Levi-21545/api-fuzzer`

não documentado recebe `MEDIUM` e erro 5xx ou ausência de resposta recebe `HIGH`. As duas abordagens recebem as mesmas famílias como entrada, e cada família recebe o maior escore entre os seus membros, como em uma triagem. A análise registra a matriz de confusão, a acurácia, a acurácia balanceada (média da taxa de acerto de cada classe), o macro-F1, o erro ordinal médio e a correlação de Spearman entre o escore e a faixa de referência, codificando `LOW`, `MEDIUM` e `HIGH` como 0, 1 e 2. O erro ordinal é a média das diferenças absolutas entre a faixa prevista e a faixa de referência. No Meilisearch, o macro-F1 e a acurácia balanceada consideram apenas as classes presentes, pois a referência não teve famílias `HIGH`. Famílias indeterminadas são contabilizadas separadamente e não entram nessas métricas. Cada família contribui uma única vez para a comparação.

Na Etapa 2 não houve nova referência cega nem reaproveitamento dos rótulos da Etapa 1. A coleta avalia execução, deduplicação e estabilidade, e a distribuição das prioridades é apenas descrita, sem métricas de acerto. O FIS não foi recalibrado.

Na coleta final, a estabilidade é medida pela presença de um mesmo ID semântico nas cinco campanhas, sem uma reexecução adicional de cada requisição. Cada ID conta no máximo uma vez por campanha e é considerado estável quando aparece em pelo menos quatro delas sob a mesma configuração. A proporção estável usa como denominador a união dos IDs das cinco campanhas da mesma aplicação.

A redução por deduplicação compara o total de achados brutos com a soma das assinaturas estritas das campanhas: $1-U/R$, em que R representa os achados brutos e U as assinaturas estritas. A união de IDs semânticos é outra medida, usada para observar presença entre campanhas, e não substitui U nessa fórmula. Na descrição do FIS, cada assinatura estrita recebe o maior escore dos seus casos, a mesma regra usada para as famílias da Etapa 1. Casos com a mesma resposta podem ter prioridades diferentes porque o FIS também considera a requisição (se deveria ser rejeitada, se foi enviada sem credencial) e a evidência reunida entre os casos.

4.4. Limitações e Ameaças à Interpretação

O que é medido. O contrato OpenAPI funciona como uma referência parcial: ele descreve o comportamento esperado, mas pode omitir regras existentes na implementação. A ferramenta também interpreta apenas o subconjunto de OpenAPI que suporta. Por isso, uma divergência encontrada é um comportamento que merece inspeção, não uma prova de defeito ou vulnerabilidade. O escore do FIS ordena a triagem e não estima impacto real, explorabilidade ou gravidade de segurança.

Condições da execução. Estado residual, ordem das requisições, latência da máquina e falhas internas da aplicação podem mudar o resultado de uma campanha. A reinicialização entre campanhas, concorrência 1, atraso fixo e manifestos reduzem essas diferenças. Ainda assim, não é possível garantir que toda variação do ambiente tenha sido eliminada.

Alcance das aplicações. O estudo usa cinco aplicações: quatro participaram do desenvolvimento e dos pilotos, e o Meilisearch foi mantido como aplicação externa para confirmar a priorização. Esse desenho permite testar o modelo fora do conjunto de desenvolvimento que o originou, mas

não representa a variedade de todas as APIs REST. As conclusões ficam limitadas às aplicações, contratos, versões e configurações avaliados.

Força das conclusões. O número de famílias e a distribuição entre `LOW`, `MEDIUM` e `HIGH` limitam algumas métricas. A referência de prioridade foi produzida às cegas por instâncias de um mesmo modelo de linguagem, cujos julgamentos são correlacionados, e não por especialistas humanos. O trabalho também não mede cobertura de código.

5. Desenvolvimento da Ferramenta

O OpenAPI Contract Fuzzer foi implementado em Python como um fluxo modular. A organização separa interpretação do contrato, geração dos casos, execução HTTP, análise das respostas, priorização e persistência dos artefatos. Esta seção descreve as responsabilidades necessárias para compreender e reproduzir o fluxo.

5.1. Do Contrato ao Plano de Execução

O analisador aceita documentos OpenAPI em YAML ou JSON e os converte em um modelo interno de operações, parâmetros, corpos, respostas e requisitos de segurança. O subconjunto implementado reconhece GET, POST, PUT, PATCH e DELETE; parâmetros de caminho, consulta e cabeçalho; corpos e respostas JSON; referências locais; e composição limitada por `allOf`. Restrições como obrigatoriedade, `readOnly`, enumeração, formato, padrão, tamanhos e limites numéricos são preservadas quando suportadas.

O gerador produz a entrada-base e o motor de mutações percorre sua estrutura para aplicar alterações tipadas. O construtor do plano resolve caminhos, corpos, cabeçalhos, autenticação e estratégia aplicada. Cada entrada recebe um `case_id` e registra se representa caso-base, mutação ou teste de autenticação, além dos metadados das mutações. O plano completo é persistido antes das chamadas à API.

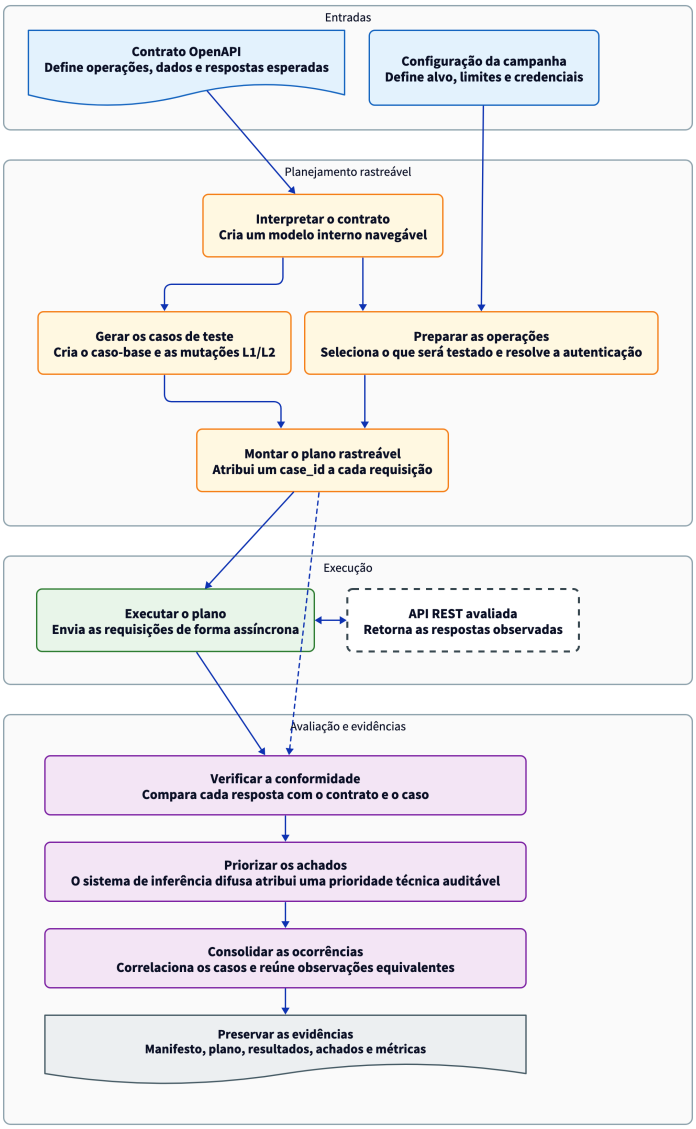


Figura 2. Arquitetura interna do OpenAPI Contract Fuzzer. Contrato e parâmetros são transformados em um plano rastreável; as respostas da API são confrontadas com o comportamento documentado, priorizadas pelo sistema *fuzzy* e consolidadas em artefatos auditáveis.

5.2. Execução, Análise e Rastreabilidade

A execução utiliza cliente HTTP assíncrono, semáforo de concorrência, atraso e tempo limite configuráveis. Para cada caso são registrados URL, status, tempo, tamanho, corpo e erro de transporte. Os executores experimentais cuidam da inicialização, da reinicialização e da proveniência específica das aplicações sem alterar o núcleo da ferramenta.

O analisador confronta status e esquemas observados com o contrato. Entre as classificações estão falhas de transporte, erros 5xx, status não documentados, incompatibilidades de esquema e aceitação de entradas que deveriam ser rejeitadas. Uma rejeição 4xx documentada e compatível é preservada como comportamento esperado e separada dos candidatos. Uma

divergência contratual continua relevante mesmo quando a API rejeita a entrada com segurança, pois pode afetar consumidores.

O pós-processamento correlaciona plano, resultado e achado por `case_id`, constrói assinaturas, seleciona representantes e calcula métricas. Cada diretório de campanha reúne `run_manifest.json`, plano de execução, resultados brutos, achados estruturados, conjunto deduplicado e relatórios em Markdown. O manifesto preserva configuração, revisão e estado do código, resumos criptográficos do contrato e da árvore-fonte, ambiente, horários, contagens e estado da campanha. Cabeçalhos e valores sensíveis são redigidos antes da persistência pública.

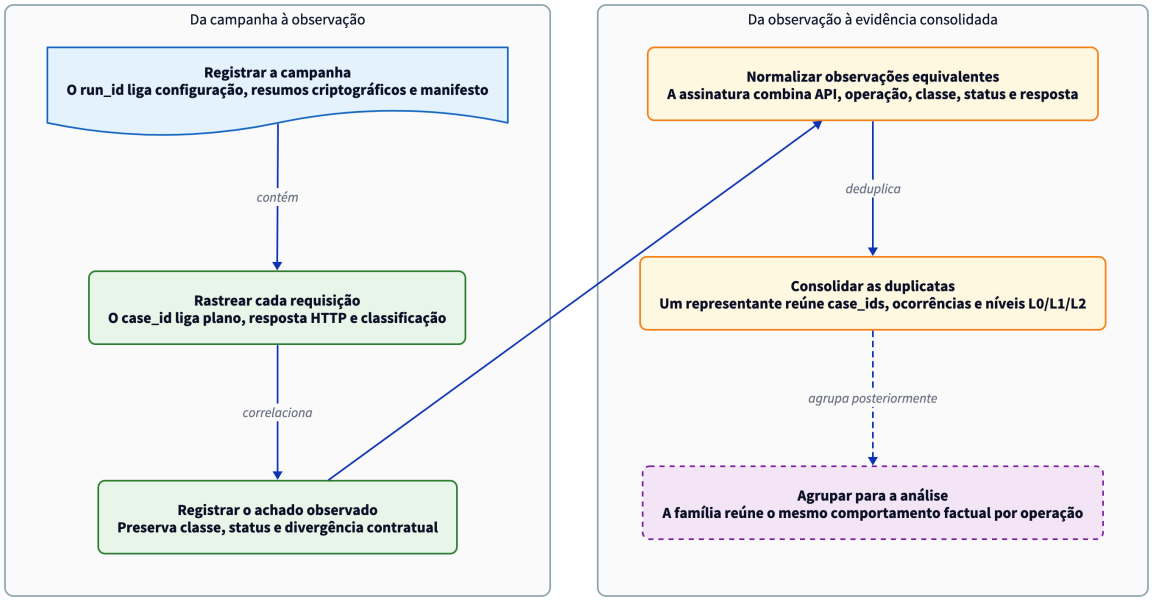


Figura 3. Níveis de observação e rastreabilidade dos achados. O `case_id` relaciona plano, resposta e classificação; a deduplicação produz achados operacionais únicos. O agrupamento posterior em famílias pertence à avaliação da priorização (Etapa 1); na coleta final, a comparação entre campanhas utiliza IDs semânticos.

Esse conjunto permite rastrear os números do artigo até casos concretos e distinguir falha da aplicação, falha ambiental e campanha incompleta. As 25 campanhas da coleta final estão publicadas no repositório do projeto, em `experiments/tcc-2026/final/`, com os cinco contratos e suas licenças, um relatório de resultados e um arquivo CSV com um ID semântico por linha; só os registros de execução dos contêineres foram omitidos. O pacote inclui o comando que valida as campanhas e reproduz a análise sem as aplicações em execução.

5.3. Priorização *Fuzzy*

Depois da classificação contratual, um sistema Mamdani combina sinais observáveis para produzir um escore contínuo e as faixas `LOW`, `MEDIUM` e `HIGH`. O FIS não decide se existe divergência; ele organiza a inspeção dos achados já reportados. Entradas, pertinências, regras ativadas e eventual regra alternativa são preservados nos artefatos para tornar a decisão auditável.

O construto é prioridade técnica da anomalia, abrangendo *contract drift*, compatibilidade funcional, robustez e controle de acesso. A evolução do modelo e os sinais utilizados em sua

revisão são apresentados uma única vez na Seção 6.2, junto das evidências que motivaram a mudança.

A cobertura de OpenAPI permanece parcial. Outros tipos de mídia, `oneOf`, `anyOf`, referências externas e parte das regras de serialização não são suportados. A ferramenta também não resolve dependências gerais entre recursos ou sequências com estado; identificadores dinâmicos podem apontar para recursos inexistentes. Esses limites restringem as operações exercitáveis e acompanham a interpretação dos resultados.

6. Resultados e Discussão

Os resultados seguem as duas etapas da metodologia. Na Etapa 1, os pilotos caracterizaram o fluxo e expuseram limitações do mecanismo de priorização, a auditoria orientou sua revisão e o modelo congelado foi validado em uma API que não participou do desenvolvimento. Na Etapa 2, a coleta final avaliou execução, deduplicação e estabilidade nas cinco aplicações.

6.1. Caracterização dos Pilotos e dos Níveis de Mutação

As campanhas de Nível 2 contra VAmPI, Spring Petclinic REST, crAPI e RealWorld executaram 958 de 958 requisições. Desse total, 284 eram casos L2 com duas mutações; os demais preservaram casos-base e mutações isoladas. A deduplicação produziu 265 assinaturas estritas, reunidas em 161 famílias comportamentais. A referência cega distribuiu essas famílias em 99 `LOW`, 59 `MEDIUM` e 3 `HIGH`.

O Nível 0 forneceu a observação nominal das operações; o Nível 1 isolou o efeito dos operadores; e o Nível 2 ampliou a variedade de manifestações, especialmente na crAPI. As combinações não produziram uma nova família `HIGH` exclusiva, mas revelaram *contract drift*, respostas incompatíveis e variações de robustez que também fazem parte do objeto.

6.2. Auditoria e Revisão do Modelo de Priorização

A configuração inicial do FIS combinava desvio de status HTTP, tempo de resposta e tamanho da resposta. A avaliação cega mostrou que o status dominava as decisões: neutralizar tempo ou tamanho não alterou a faixa de nenhuma das 161 famílias. O modelo e a linha de base baseada somente em status obtiveram a mesma acurácia, 0,298, o mesmo macro-F1, 0,254, e o mesmo erro ordinal médio, 0,702.

A matriz de confusão revelou 67 famílias `LOW` elevadas a `MEDIUM` e 43 famílias `MEDIUM` elevadas a `HIGH`. O mecanismo superpriorizava rejeições seguras, 4xx não documentados e falhas 5xx localizadas. Uma rejeição não documentada pode representar *contract drift*, mas não deve receber automaticamente a mesma prioridade de uma operação válida indisponível, de uma aceitação indevida ou de uma falha recorrente.

Esse diagnóstico levou à revisão do construto e dos antecedentes. O modelo revisado utiliza contradição comportamental, força da evidência e exposição de controle de acesso. A contradição representa a incompatibilidade entre o caso, o status, o esquema e o comportamento observado; a evidência distingue manifestações isoladas de determinadas recorrências; e a exposição de acesso captura operações protegidas que aceitam credencial ausente ou inválida. Seis regras Mamdani agregam esses sinais e produzem o escore contínuo.

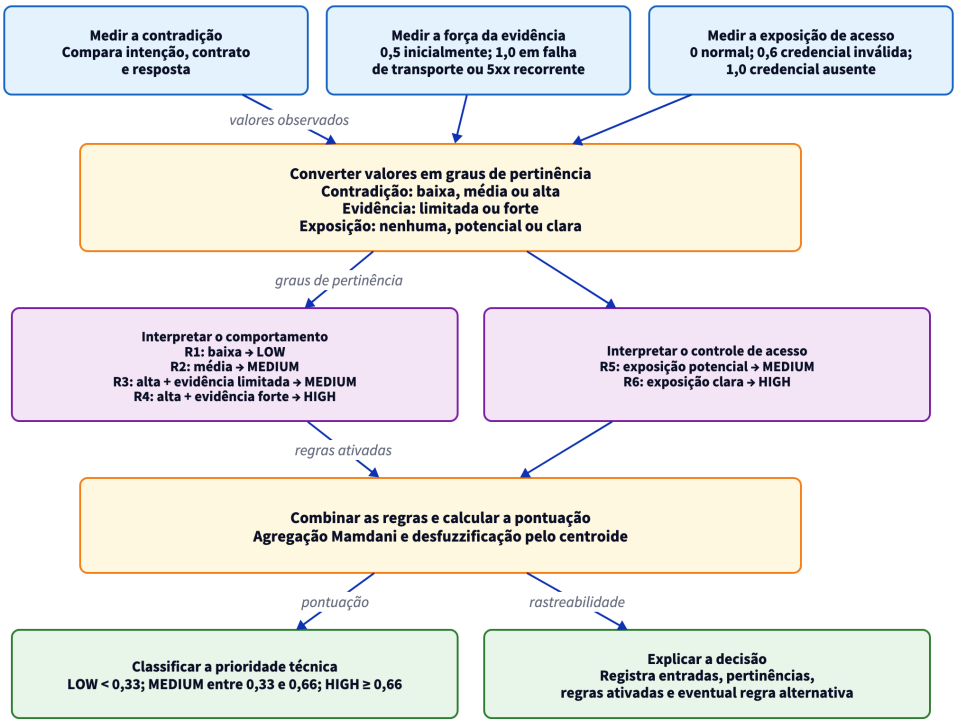


Figura 4. Fluxo auditável do sistema Mamdani de priorização. Sinais do contrato, da resposta e da autenticação são fuzzificados e combinados por seis regras. A saída representa prioridade técnica, não confirmação de vulnerabilidade.

No conjunto de desenvolvimento, o modelo revisado reproduziu as 161 faixas da referência, com macro-F1 1,000 e erro ordinal zero. Esse resultado demonstra que a implementação expressa os critérios formulados, mas não constitui evidência de generalização, pois a mesma amostra foi usada para diagnosticar e revisar o mecanismo. Por isso, regras e parâmetros foram congelados antes da avaliação externa.

6.3. Validação Externa no Meilisearch

O modelo congelado foi aplicado ao Meilisearch v1.45.1 em cinco campanhas, cada uma com 85 operações selecionadas e 477 requisições concluídas. As assinaturas estritas apresentaram interseção 216 e união 276. A variação veio dos campos `requestUid` e `processingTimeMs` em `POST /multi-search`. Após normalização restrita a esses dois campos, cada repetição apresentou as mesmas 222 assinaturas semânticas.

As assinaturas foram agrupadas em 134 famílias. A referência final classificou 122 como sustentadas e 12 como indeterminadas. Entre as sustentadas, 116 receberam `LOW`, 6 receberam `MEDIUM` e nenhuma recebeu `HIGH`.

O modelo revisado concordou com a referência em 120 das 122 famílias sustentadas; a linha de base baseada somente em status acertou 4. A diferença vem sobretudo das 116 famílias `LOW`: a linha de base atribuiu `MEDIUM` a todas, porque seus status não estavam documentados no

contrato, o que também explica sua correlação negativa. A Tabela 2 apresenta as métricas nas classes observadas.

Tabela 2. Comparação nas 122 famílias sustentadas do Meilisearch

Métrica	Modelo revisado	Somente status
Acurácia	0,9836	0,0328
Macro-F1 (classes observadas)	0,8957	0,0317
Acurácia balanceada	0,8333	0,3333
Erro ordinal médio	0,0164	0,9672
Spearman	0,9088	-0,6981

Como 84 das 122 famílias sustentadas eram controles de credencial inválida, foi realizada uma análise sem esses casos. O modelo revisado permaneceu correto em 36 de 38 famílias, contra 4 de 38 da linha de base. Os dois desacordos foram preservados: um caso-base de `PATCH /experimental-features` com status 415 não documentado e uma mutação válida de `POST /export` que retornou 200 quando o contrato documentava 202. O FIS não foi reajustado com esses casos.

6.4. Coleta Final nas Cinco Aplicações

As 25 campanhas finais concluíram as 7.175 requisições planejadas, e a validação do pacote não identificou campanhas inválidas. Foram observadas 760 respostas 2xx, 5.450 respostas 4xx e 965 respostas 5xx. Esses números descrevem as respostas aos casos gerados, incluindo mutações negativas; não constituem uma taxa de defeitos das APIs.

A Tabela 3 apresenta as contagens de execução e deduplicação. As 2.605 rejeições esperadas são respostas 4xx documentadas e compatíveis com o contrato, separadas dos 4.000 achados brutos. A redução de 38,5% compara esses achados com as 2.460 assinaturas estritas somadas nas campanhas.

Tabela 3. Execução e deduplicação na coleta final, somando cinco campanhas por API

API	Requisições	Rejeições esperadas	Achados brutos	Assinaturas estritas	Redução
VAmPI	245	160	35	35	0,0%
Petclinic REST	1.780	1.155	590	235	60,2%
crAPI	2.055	260	1.720	985	42,7%
RealWorld	710	610	65	65	0,0%
Meilisearch	2.385	420	1.590	1.140	28,3%
Total	**7.175**	**2.605**	**4.000**	**2.460**	**38,5%**

As diferenças de redução dependem do contrato e dos comportamentos observados. No Petclinic REST, várias requisições convergiram para as mesmas respostas normalizadas, enquanto VAmPI e RealWorld não apresentaram duplicatas estritas dentro das campanhas. Uma redução nula não indica ausência de repetição entre campanhas: a deduplicação estrita é calculada separadamente em cada execução. Os totais também refletem números diferentes de operações e caminhos excluídos e não comparam a qualidade das aplicações.

Para a análise entre campanhas, a união dos IDs semânticos totalizou 570. A Tabela 4 conta cada ID no máximo uma vez por campanha. Dos 570 IDs, 441 apareceram nas cinco repetições da respectiva API, 12 em quatro, 18 em três, 24 em duas e 75 em apenas uma. Pelo critério de pelo menos quatro campanhas, 453 IDs atenderam à definição operacional de estabilidade.

Tabela 4. Presença dos IDs semânticos em exatamente k das cinco campanhas por API

API	IDs	5/5	4/5	3/5	2/5	1/5
VAmPI	11	6	0	0	0	5
Petclinic REST	47	47	0	0	0	0
crAPI	273	154	12	18	24	65
RealWorld	17	12	0	0	0	5
Meilisearch	222	222	0	0	0	0
Total	**570**	**441**	**12**	**18**	**24**	**75**

Petclinic REST e Meilisearch tiveram todos os IDs presentes nas cinco campanhas. A proporção presente em pelo menos quatro foi de 54,5% em VAmPI, 60,8% na crAPI e 70,6% no RealWorld. Na crAPI concentraram-se 65 dos 75 IDs observados uma única vez, 47 deles em `POST /identity/api/auth/signup`. Em VAmPI e RealWorld, os IDs vistos uma única vez vêm de uma só operação em cada API e surgem um por campanha: a resposta inclui o nome do usuário criado naquela campanha, o que altera o ID a cada execução, sem mudança de comportamento.

A Tabela 5 descreve a prioridade atribuída, usando o maior escore dos casos de cada assinatura estrita em cada campanha. Ao todo, 1.965 assinaturas receberam `LOW`, 480 receberam `MEDIUM` e 15 receberam `HIGH`. Em 16 das 2.460 assinaturas coexistiram casos de faixas diferentes; três IDs semânticos da crAPI variaram de faixa entre campanhas. Essas diferenças são compatíveis com entradas distintas para o FIS, que considera a requisição e a evidência além da resposta. Achados com as mesmas entradas do FIS receberam sempre o mesmo escore no conjunto analisado.

Tabela 5. Prioridade por assinatura estrita e campanha na coleta final

API	`LOW`	`MEDIUM`	`HIGH`	Mediana	Máximo
VAmPI	30	5	0	0,16	0,50
Petclinic REST	125	100	10	0,17	0,84
crAPI	790	190	5	0,16	0,73
RealWorld	0	65	0	0,50	0,50
Meilisearch	1.020	120	0	0,16	0,50

Como não houve referência cega na Etapa 2, essas faixas descrevem o comportamento do modelo e não permitem calcular sua acurácia.

6.5. Discussão e Limitações

Na Etapa 1, o principal resultado foi explicar por que o modelo inicial não acrescentava nada ao status HTTP. A revisão substituiu sinais fáceis de medir, mas pouco discriminativos, por fatos mais próximos da prioridade técnica, e a validação no Meilisearch indica que essa mudança se manteve coerente em dados que não participaram do ajuste.

Essa validação tem limites: a referência não teve famílias `HIGH`, as instâncias do modelo concordaram pouco entre si sobre a prioridade e a maioria das famílias eram controles de credencial inválida. A referência é operacional, não uma verdade universal.

Na Etapa 2, todas as campanhas foram concluídas e a deduplicação reduziu as observações repetidas, mas a estabilidade variou entre aplicações. O pacote permite conferir a análise sem os ambientes originais; repetir a coleta, porém, exige reconstruir os alvos e pode produzir respostas diferentes.

Em relação à questão de pesquisa, os resultados sustentam a organização de evidências para triagem nas condições estudadas, com alcance limitado pelos recortes dos contratos, pela ausência de sequenciamento com estado e pela falta de confirmação manual dos novos achados. O efeito sobre o tempo de trabalho de um analista não foi medido.

7. Conclusão

Este trabalho investigou quais contribuições e limitações a combinação de *fuzzing black-box* orientado por OpenAPI, deduplicação e priorização *fuzzy* apresenta para a triagem de divergências em APIs REST. Para isso, foi implementado um fluxo que interpreta um subconjunto de OpenAPI, gera casos-base e mutações, executa as requisições, identifica divergências, agrupa observações equivalentes, atribui prioridade técnica e preserva os artefatos de cada campanha.

Nas três dimensões avaliadas, os resultados indicam:

- **execução e completude:** as 25 campanhas da coleta final concluíram as 7.175 requisições planejadas, e as contagens do artigo podem ser rastreadas até os planos, as respostas e os achados publicados;
- **estabilidade:** 441 dos 570 IDs semânticos reapareceram nas cinco repetições da respectiva API e 453 em pelo menos quatro. A presença foi completa no Petclinic REST e no Meilisearch; em VAmPI e RealWorld, a variação veio de dados gerados em cada campanha, e a crAPI concentrou a maior parte dos IDs observados uma única vez;
- **deduplicação e prioridade:** o agrupamento reduziu 4.000 achados brutos a 2.460 assinaturas estritas (38,5%). Na Etapa 1, o FIS revisado concordou com a referência cega em 120 de 122 famílias do Meilisearch, contra 4 da linha de base baseada somente no status.

A principal contribuição é um processo rastreável que transforma um contrato OpenAPI em evidências organizadas para triagem de *contract drift*, incompatibilidades funcionais, problemas de robustez e indícios de controle de acesso, com uma prioridade cujos critérios podem ser auditados. Os resultados não confirmam vulnerabilidades nem estimam impacto real. As conclusões são limitadas pelo número de aplicações, pelos recortes dos contratos, pelo OpenAPI como oráculo parcial e por uma referência de prioridade produzida por um modelo de linguagem, sem casos `HIGH` no Meilisearch. Na coleta final, os escores têm interpretação descritiva.

Como trabalhos futuros, ficam a ampliação das aplicações e dos recursos de OpenAPI suportados, a resolução de dependências entre operações, a avaliação da triagem por especialistas humanos, a integração com uma esteira de integração contínua e a comparação com outras ferramentas sob um mesmo protocolo.

8. Referências

Alakeel, A. M. (2014). Using fuzzy logic in test case prioritization for regression testing programs with assertions. *The Scientific World Journal*, 2014.

Atlidakis, V., Godefroid, P., e Polishchuk, M. (2019). Restler: Stateful rest api fuzzing. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*, pages 748–758.

Corradini, D., Zampieri, A., Pasqua, M., e Ceccato, M. (2021). Empirical comparison of black-box test case generation tools for restful apis. In *2021 IEEE 21st International Working Conference on Source Code Analysis and Manipulation (SCAM)*, pages 226–236.

Corradini, D., Zampieri, A., Pasqua, M., e Ceccato, M. (2022). Resttestgen: An extensible framework for automated black-box testing of restful apis. In *2022 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 504–508.

Elbaum, S., Malishevsky, A. G., e Rothermel, G. (2002). Test case prioritization: A family of empirical studies. *IEEE Transactions on Software Engineering*, 28(2):159–182.

Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine.

Fielding, R. T., Nottingham, M., e Reschke, J. (2022). Http semantics. RFC 9110.

Golmohammadi, A., Zhang, M., e Arcuri, A. (2023). Testing restful apis: A survey. *ACM Transactions on Software Engineering and Methodology*, 33(1).

Hatfield-Dodds, Z. e Dygalo, D. (2021). Deriving semantics-aware fuzzers from web api schemas. Apresentado como pôster na ICSE 2022.

JSON Schema (2022). Json schema 2020-12.

Mamdani, E. H. e Assilian, S. (1975). An experiment in linguistic synthesis with a fuzzy logic controller. *International Journal of Man-Machine Studies*, 7(1):1–13.

Martin-Lopez, A., Segura, S., e Ruiz-Cortes, A. (2020). Restest: Black-box constraint-based testing of restful web apis. In *Service-Oriented Computing*, volume 12571 of *Lecture Notes in Computer Science*, pages 459–475.

Meyer, B. (1992). Applying "design by contract". *IEEE Computer*, 25(10):40–51.

OpenAPI Initiative (2021). Openapi specification 3.1.0.

Zadeh, L. A. (1965). Fuzzy sets. *Information and Control*, 8(3):338–353.

Zhang, M. e Arcuri, A. (2023). Open problems in fuzzing restful apis: A comparison of tools. *ACM Trans. Softw. Eng. Methodol.*, 32(6).